

OMMS DISSERTATION - HOMOTOPY TYPE THEORY WITH COQ

CANDIDATE NUMBER 1102625

1. INTRODUCTION

Type theory is a foundational framework for mathematics and computer science, offering an alternative to set theory by treating mathematical objects as *types*. These types can be seen as a set, a proposition (whose elements are proofs), or, in *Homotopy* Type Theory, a topological space. This work follows the approach developed in the Homotopy Type Theory book by the Univalent Foundations Program[5].

The central feature of Homotopy Type Theory is its *internal* notion of equality, which is itself a type. These equality types are not, as in first order logic, merely predicates: they carry content that can be interpreted homotopically – we can view proofs that two elements are equal as paths between them, and define operations between paths that make each type into an ∞ -groupoid, or a *homotopy type*.

To represent the homotopy types we are used to from topology, we enrich the theory with Voevodsky’s Univalence Axiom. This axiom has profound consequences: it makes precise the usual mathematical practice of identifying isomorphic structures and forces some types to possess non-trivial paths, analogous to the circle’s non-contractible loop in topology. In fact, we will define a type-theoretic version of the circle and compute its fundamental group.

Another advantage of Homotopy Type Theory is its computational nature: it is developed within a formal deductive system and all elements and types can be reduced by rules to a standard form that can be verified by a computer programme. The interactive theorem prover Coq¹ is one such programmes, and is suitable for formalizing Homotopy Type Theory. Much of the formalization is already in the Coq-HoTT library[1]. In this work, we introduce theorem proving in Coq and present results in informal and formal terms. Most proofs here already exist in the library, with the exceptions being 6.3 and the univalence-dependent circle results in 7.2. These are in the process of being submitted. Other proofs have been reworked to use fewer auxiliary results and make this work more self-contained.

2. PRIMITIVE OBJECTS

Although this work is about informal type theory, it is useful to have a broad notion of how what we are doing can fit in a constructive formal setting, i.e., a deductive system.

Whenever we are doing any mathematics, we have a collection of symbols to express it. We use variable symbols (for example “ x ” in the expression $x \mapsto f(x)$); syntactic symbols, such as “(”, “:”, “=”, “ $\equiv$ ”, “ $\mapsto$ ”, to which we have assigned a precise meaning, and also constant symbols, which we may use to abbreviate expressions, like “ f ” in $f \equiv x \mapsto \Phi$.

Once we have this collection of symbols, we can form *judgements* using *rules*. These judgements are added to the *context*. Once a judgement is in the context, we can apply rules to derive further judgements.

Our judgements are of two kinds. The first, $a : A$, reads “ a is an element of type A ”, and all the theorems we’ll prove will be of this form.

We will also have judgements that look like $a \equiv b$ (which reads a is judgementally equal to b). This only makes sense in the context of judgements $a : A$ and $b : A$. Thus, a judgemental equality is never derived on its own, but as a *computational rule* associated to some other judgement(s) of the former kind, that tell us how terms we’ve constructed behave.

Not all type theories have judgemental equality as a feature. Those that do are called *intensional* (as opposed to *extensional*) type theories. The rules associated to judgemental equality make it like the equality we are used to: it is symmetric, transitive, and we can perform δ -reductions [4,

¹Coq is currently undergoing a re-brand to Rocq. However, this is still not prevalent.

Conversion Rules], i.e., replace a term in a judgement by a term judgementally equal to it. In other words, it is computable and Coq can unfold it automatically.

Judgemental equality can also be used to define symbolic abbreviations for expressions, which, in keeping with the naming principles in the Coq-HoTT library, will often be descriptive and consist of multiple words separated by underscores.

2.1. Universes. How do we express that something is a type? If we only want to have statements that look like $A : B$, then the best way is to define a *type of types*, i.e. a universe. If we have such a universe, $\mathcal{U}$, we can write $A : \mathcal{U}$ to mean that A is a type. Universes are part of the definition of a type theory: they are a starting point for us to be able to write down statements.

Of course, if we are not careful when defining this notion, we will inadvertently create Russell-like paradoxes[2, page 6]. One approach is to define *cumulative* universes [5, Section 1.3], indexed by ordinals, $\mathcal{U}_0 \subseteq \mathcal{U}_1 \subseteq \mathcal{U}_2 \subseteq \dots$, such that $\mathcal{U}_i : \mathcal{U}_{i+1}$. We won't get into the details of this, and in fact we will very rarely write the index of the universe we are working with. In Coq, we can simply write **Type** for $\mathcal{U}$, without any index, and it will automatically work with the smallest universe that makes sense.

2.2. Function Types. Another part of the definition of this deductive system is the function type former. This consists of a collection of rules that will allow us to form new judgements from pre-existing ones.

Definition 2.2.1. Given two judgements $A : \mathcal{U}$ and $B : \mathcal{U}$, i.e., given two types A and B , we can form the judgement $A \rightarrow B : \mathcal{U}$, i.e., we can form the *function type* $A \rightarrow B$, i.e., the judgement $A \rightarrow B : \mathcal{U}$ ²

Clearly, this definition is incomplete: we need to give more information about how a function type works if we want to be able to use functions like we do in everyday mathematics. This will come in the form of some additional rules. This list is an example of a very general construction in type theory, where a type former is described via its

- (1) **formation rule(s)**: It forms a new type (possibly from pre-existing types) and gives it a name. In the case of functions, it corresponds to definition 2.2.1 given above.
- (2) **constructor(s)**: also called an *introduction rule*, it provides elements of the type being formed. For functions, it is called *lambda introduction*, and states that if our context, together with the momentary assumption $x : A$, can derive the judgement $\Phi : B$ (where Φ is a term possibly involving the variable x), then we can conclude $(\lambda x. \Phi) : A \rightarrow B$.
- (3) **eliminator(s)**: the elimination rule tells us how to *deconstruct* elements of the type being formed, usually to get elements of the type(s) we've started with. For functions, this corresponds to *function application*: if we have $f : A \rightarrow B$ and $a : A$, then we have $f a : B$.
- (4) **computation rule(s)**: stating that we can apply a function to an element is insufficient: we must say how to compute the value of a function given by a certain expression, i.e., *how the eliminator acts on the constructor*. This is called a *computation rule*, which yields a judgemental equality. Naturally, for function types this rule takes the same premise as the constructor, together with $a : A$, and concludes $((\lambda x. \Phi) a) \equiv \Phi\{x \backslash a\}$ ³.
- (5) **uniqueness principle**: This optional rule can, in some cases, be seen as a dual to the computation rule, i.e., it is a judgemental equality expressing how *constructors act on eliminators*. For functions, it takes the premise $f : A \rightarrow B$ and produces the conclusion $f \equiv \lambda x. (f x)$.

²if $A : \mathcal{U}_j$ and $B : \mathcal{U}_k$ then $A \rightarrow B : \mathcal{U}_{k \vee j}$

³ $\Phi\{x \backslash a\}$ stands for the term obtained from Φ when we replace all (free) occurrences of the variable symbol x by the term a

An important class of function types are type families. A type family (over a type A) is simply an element of type $A \rightarrow \mathcal{U}_i$ for some ordinal i (note that then the type of type families is an element of $\mathcal{U}_{i+1}$).

Once we have a type family $B : A \rightarrow \mathcal{U}$, we can form the type of dependent functions on B , which we denote by $\prod_{x:A} B x$. Its definition is identical to that of the type $A \rightarrow B$, but with one key difference: in the constructor we demand $\Phi : B x$ when $x : A$ (instead of $\Phi : B$). This means that an element of type $\prod_{x:A} B x$ will behave like a function with domain A , but where the co-domain depends on the input.

An important example is the type $(\prod_{X:\mathcal{U}} (X \rightarrow X)) : \mathcal{U}$, along with an element $(\lambda X. (\lambda x. x)) : (\prod_{X:\mathcal{U}} X \rightarrow X)$. From a categorical perspective, this type would be $\mathbf{End}(_)$ and the element $\mathbf{id}__$.

In Coq, we write $A \rightarrow B$ for the type $A \rightarrow B$. We can also write $\mathbf{forall} \ (x : A), B \ x$ to mean $\prod_{x:A} B x$. The constructor of (dependent) function types, $\lambda x. \mathbf{expr}$, is written $\mathbf{fun} \ x \Rightarrow \mathbf{expr}$. For example, we could write the type of endo-functions $(\prod_{X:\mathcal{U}} X \rightarrow X) : \mathcal{U}$ as $\mathbf{forall} \ (X : \mathbf{Type}) \ X \rightarrow X$. To introduce the identity element, we write

Definition $\mathbf{id} : \mathbf{forall} \ (X : \mathbf{Type}), X \rightarrow X := \mathbf{fun} \ X \Rightarrow (\mathbf{fun} \ x \Rightarrow x)$.

In Coq, applying a function to an argument is done by juxtaposition (hence why we are using that convention in this document). The computation rule corresponds to what is called β -reduction [4, Conversion Rules], which allows Coq to simplify the term $(\mathbf{fun} \ x \Rightarrow \mathbf{t}) \ u$, and the uniqueness principle corresponds to η -expansion [4, Conversion Rules].

Notice the keyword **definition** used above. This is used whenever we want to give a name to a specific element of a type (in this case “ $\mathbf{id}$ ”). The colon introduces the type of the element being introduced, and $:=$ introduces the rules that we are using to build the element (λ -introduction twice). If the syntax is well typed, Coq checks if the element we described is of the desired type, and if so adds this judgement to our context. These definitions are automatically unfolded when performing other type checks.

Coq also accepts $\mathbf{fun} \ X \ x \Rightarrow x$ as a shortcut for $\mathbf{fun} \ X \Rightarrow (\mathbf{fun} \ x \Rightarrow x)$. This is a common practice in Type Theory: in order to define a function with multiple arguments, we define it iteratively, as a function into a function type. This is called “currying”, after Haskell Curry. To make this easy to write, (dependent) function types are parsed on the right, i.e. $A \rightarrow B \rightarrow C$ means $A \rightarrow (B \rightarrow C)$.

A key idea in type theory is that any definition can be seen as a theorem and proof. The type (here $\mathbf{forall} \ (X : \mathbf{Type}), X \rightarrow X$) is the statement to be proved, and the element specified ($\mathbf{fun} \ X \ x \Rightarrow x$) is the proof.⁴

This is the famous “Propositions-as-types” correspondence [5, Section 1.11]. Under this correspondence, the statement reads “every proposition implies itself”, and the proof reads “take a proposition X . Suppose X has a proof, say x . Then we can produce a proof of X , namely x ”.

With the tools that we have, we can build proofs of tautologies (like $\mathbf{id}$), but we can’t do much else – we need a way to introduce concrete types, as well as more ways to construct new types out of pre-existing ones (for example by taking products or co-products). All of these constructions fit into a general framework: that of inductive types.

3. INDUCTIVE TYPES

Inductive types in type theory were first introduced by Martin-Löf [3] in the late twentieth century.

⁴in this interpretation, $A \rightarrow B$ means “ A implies B ”, since to define such a function is to give a way of turning a proof of A into a proof of B

These types are *presented* by constructors, and characterized by the rules for *mapping out* of them (in this way they are similar to categorical co-limits). The general slogan is that in order to define a function out of an inductive type, we only need to say what it does on the constructors: we call this the induction principle for that inductive type.

We will see that inductive types are extremely powerful, and will allow us to define the notion of internal equality.

The full systematic definition of an inductive type is complex, and can be found in [4, Inductive types]. This is essentially a collection of conditions for a given list of constructors to define a valid inductive type, along with rules for deriving an induction principle. If we want to define an inductive type in Coq, we simply write the keyword `Inductive` and list the constructors, the induction principle being automatically generated from them. However, inductive types are implemented slightly differently in the HoTT library. In this section, we gloss over those details and present some examples of inductive types that will be important in the rest of the text.

3.1. The Unit Type. The unit type, $\mathbf{1}$, has one constructor, $\star : \mathbf{1}$. Its induction principle is

$$\text{ind}_{\mathbf{1}} : \prod_{P : \mathbf{1} \rightarrow \mathcal{U}} \left(P \star \rightarrow \prod_{x : \mathbf{1}} P x \right)$$

It says that to construct a function out of the unit type, we need only provide an element of the co-domain. This element will be the image of $\star$ – this is expressed by the following associated computation rule

$$\text{ind}_{\mathbf{1}} P c \star \equiv c$$

In the Coq-HoTT library, this type is given the name `Unit`, and its constructor `tt`. Its induction principle is the function `Unit_ind`. As a judgemental equality, the computation rule will automatically be applied when appropriate.

3.2. Co-product Types. The co-product type former takes two types A and B and produces a type $A + B$, with the following constructors:

- $\text{inl} : A \rightarrow A + B$
- $\text{inr} : B \rightarrow A + B$

Its induction principle is

$$\text{ind}_{A+B} : \prod_{P : A+B \rightarrow \mathcal{U}} \left(\prod_{a:A} (P (\text{inl } a)) \right) \rightarrow \left(\prod_{b:B} (P (\text{inr } b)) \right) \rightarrow \prod_{x:A+B} P x$$

It comes with the computation rules

- $\text{ind}_{A+B} P f g (\text{inl } a) \equiv f a$
- $\text{ind}_{A+B} P f g (\text{inr } b) \equiv g b$

i.e., to map out of $A + B$, specify images on $\text{inl } a$ and $\text{inr } b$. In the Coq-HoTT library, `A + B` denotes the coproduct, and `inl B a`, `inr A b` denote the injections. The induction principle is written `sum_ind`.

3.3. The Booleans. We can define the type of Booleans, $\mathbf{2}$, in two ways: as the inductive type presented by two constructors, 0_2 and 1_2 , or as $\mathbf{1} + \mathbf{1}$. It is easy to see that these two constructions amount to the same: the induction principle of the first would tell us that in order to define a function out of the Booleans we need only define it on 0_2 and 1_2 , whereas if we apply the induction principles for $+$ and for $\mathbf{1}$ we need to define it on $\text{inl } \star$ and $\text{inr } \star$.

Although the former definition is more typical, we will choose to go with the latter, since it will enable us to apply results proven for coproduct types directly, without having to translate them through an equivalence (defined later in section 4.6). We will, however, use the abbreviations 0_2 and 1_2 to mean $\text{inl } \star$ and $\text{inr } \star$, respectively. In Coq, this can be realized as

`Definition Bool := Unit + Unit.`

`Definition zero2 := inl Unit tt.`

`Definition one2 := inr Unit tt.`

3.4. The Empty type. Another trivial but important inductive type is the empty type, which we will denote by $\mathbf{0}$. This type has no constructors, so its induction principle should say that in order to define a function out of the empty type we needn't provide anything. We can write it in the following way.

$$\text{ind. } \prod_{P: \mathbf{0} \rightarrow \mathcal{U}} \prod_{x: \mathbf{0}} P x$$

An element of this type is called a contradiction. If we have one, we can produce an element of any type we wish, using the induction principle. However, elements of the empty type are not easy to come by, since it has no constructors. Thus, the only way we can have a contradiction is if we start with some false premise, i.e. if we are trying to prove a statement of the form $A \rightarrow B$, where A is a type that should not have any elements. We'll do some proofs that involve the empty type in sections 6 and 7.2.

A use of the empty type is negation. Given a type A , we define its negation $\neg A$ as $A \rightarrow \mathbf{0}$, which we can interpret as “ A has no elements”. To prove $\neg A$ must produce a contradiction out of an arbitrary element of A .

3.5. Product and Dependent Pair Types. Although usually we see products defined by projections out of them, we can also define them by maps going into the product, by currying. Thus, given two types A and B , the type $A \times B$ has the constructor

$$\text{pair} : A \rightarrow B \rightarrow A \times B$$

The induction principle for products is

$$\text{ind}_\times : \prod_{A, B: \mathcal{U}} \prod_{C: A \times B \rightarrow \mathcal{U}} \left(\prod_{a: A} \prod_{b: B} C (\text{pair } a b) \right) \rightarrow \prod_{x: A \times B} C x$$

with computation rule

$$\text{ind}_\times A B C f (\text{pair } a b) \equiv f a b$$

This means that defining a function out of a product is the same as defining a curried function. Product types will not be very useful in the rest of this work. Indeed, currying is the more idiomatic way to define multi-argument functions in type theory.

More interesting, perhaps, are dependent pair types, also called sigma types. These are similar to products, but the type of the second component can depend on the first. More precisely, if we have $A : \mathcal{U}$ and $P : A \rightarrow \mathcal{U}$, the type $\sum_{x:A} P x$ has constructor

$$\text{pair} : \prod_{a:A} \left(P a \rightarrow \sum_{x:A} P x \right)$$

, which is similar to the constructor for product types, except it is dependent on the first argument. The induction principle also only differs in this way:

$$\begin{aligned} \text{ind}_\Sigma : \prod_{A:\mathcal{U}} \prod_{P:A \rightarrow \mathcal{U}} \prod_{C:(\sum_{y:A} P y) \rightarrow \mathcal{U}} & \left(\prod_{a:A} \prod_{b:P a} C (\text{pair } a b) \right) \rightarrow \prod_{x:\sum_{y:A} P y} C x \\ \text{ind}_\Sigma A P C f (\text{pair } a b) & \equiv f a b \end{aligned}$$

Just as in product types, we won't use the induction principle. However, sigma types are useful for

- (1) writing existential statements : if we view $P : A \rightarrow \mathcal{U}$ as a predicate on A , i.e. view each $P a$ as a proposition, and view A as a set, the type $\sum_{x:A} P x$ can read as the proposition “there exists $x : A$ such that $P x$ ”. Note a proof of this proposition, i.e., an element of this type, consists of an element $a : A$ and a proof of $P a$;
- (2) writing sub-types : If we still view P as a predicate, but not $\sum_{x:A} P x$ as a proposition, then this type can be interpreted as the “sub-type” of A consisting of the elements that satisfy P .

3.6. Natural Numbers. The natural numbers, $\mathbb{N}$, are the only example we'll see of an inductive type that is recursive, i.e. where a constructor takes as an argument an element of the type being defined. Its constructors are:

- $0_{\mathbb{N}} : \mathbb{N}$
- $\text{succ} : \mathbb{N} \rightarrow \mathbb{N}$

The induction principle is very familiar from the theory of purely recursive functions in mathematical logic and computer science. It is as follows.

$$\text{ind}_{\mathbb{N}} : \prod_{P:\mathbb{N} \rightarrow \mathcal{U}} (P 0) \rightarrow \left(\prod_{n:\mathbb{N}} P n \rightarrow P (\text{succ } n) \right) \rightarrow \prod_{n:\mathbb{N}} P n$$

Its computation rules are:

- $\text{ind}_{\mathbb{N}} P p_0 p_s 0_{\mathbb{N}} \equiv p_0$
- $\text{ind}_{\mathbb{N}} P p_0 p_s (\text{succ } n) \equiv p_s n (\text{ind}_{\mathbb{N}} P p_0 p_s n)$

This allows us to define a (dependent) function f by recursion, i.e., by specifying a value at 0 and giving a rule for calculating $f (\text{succ } n)$ from n and $f n$. Any mathematician will also recognise this as the usual principle of induction for natural numbers, if we view P as a proposition. The main use for natural numbers in this work will be as a means to define the integers, which we will see to be the fundamental group of the circle.

3.7. Equality types. So far, we have looked at judgemental equality, which is purely computational and external to our type theory. We also want to have an *internal* notion of equality, i.e., we want to be able to express that two elements of the same type are “equal” in the form of a type-theoretic statement of the form $a : A$. Thus, we will build *equality types*.

Definition 3.7.1. If $A : \mathcal{U}$, $a : A$, then we have the type family $\text{eq}_{A,a} : A \rightarrow \mathcal{U}$, with constructor $\text{refl}_a : \text{eq}_{A,a} a$

Naturally, we will write $a =_A x$ for the type $\text{eq}_{A,a} x$, which reads as “ a is propositionally equal to x ”. This is usually called *based* equality, since the left endpoint of the equality is fixed. There is also an unbased version, which is completely equivalent [5, Section 1.12]⁵.

Elements of an equality type can be seen both as proofs that two elements are equal or, if we think of types as spaces, as paths between the two elements. The fact that equality types only have one constructor, reflexivity, suggest that the only way to have $a = b$ is if $b \equiv a$, and the only way they can be equal is through the trivial path refl_a . As things stand, there is nothing wrong with that interpretation. However, once we introduce extra axioms, like univalence and higher inductive types, we will see that it is possible to have paths between judgementally different elements, as well as non-trivial loops.

The induction principle for this type former is

$$\text{ind}_= : \prod_{A:\mathcal{U}} \prod_{a:A} \prod_{P:\prod_{x:A} a=x \rightarrow \mathcal{U}} \left((P a \text{refl}_a) \rightarrow \left(\prod_{x:A} \prod_{p:a=x} P x p \right) \right)$$

, along with the computation rule $\text{ind}_= A a P d a (\text{refl}_a) \equiv d$.

This is usually called (based) path induction, and it allows us to define a dependent function from equality types on A by specifying what the image of reflexivity is.

Above, we were very quick to drop any reference to the type A when writing $x = y$ to mean $\text{eq}_{A,x} y$. Indeed, there is no reason for keeping it, since A can be inferred from the arguments x and y , as it is their type. This is not only a feature of informal mathematics, but also a very useful mechanism implemented in Coq: when arguments of a dependent function can be inferred from other arguments that depend on them, we can make them *implicit*. We do this by using curly braces instead of parentheses. When applying the function, we do not need to write the implicit arguments, and Coq will attempt to infer them automatically. If there is not enough information, we will receive an error message.

Theorem 3.7.2. Let A be a type, x, y elements of A . If we have a path from x to y , then we also have a path from y to x , i.e., “equality is symmetric”.

Proof. This statement, expressed in an informal style, can be translated into the type `forall (A : Type) (x y : A), x = y -> y = x`. Proving the theorem is the same as defining an element of this type. This can be done using the induction principle for equality types, in the following way.

We want to construct a dependent function over the type family `fun A x y p => y = x`. Fixing A and x , to use the induction principle, we need to provide an element of $(\lambda y p. y = x) x (\text{refl}_x)$, which by β -reduction is judgementally $x = x$. But we have an element of this type, namely refl_x . Putting this all together, we obtain the following formalized proof:

```
Definition inv : forall {A : Type} {x y : A}, x = y -> y = x :=
fun A x => paths_ind x (fun y p => y = x) (reflexivity x).
```

⁵we chose to present the based version because it is the one implemented in the Coq-HoTT library

□

Theorem 3.7.3. *Let A be a type, x, y, z elements of A , and suppose we have a path from x to y and another from y to z . Then we have a path from x to z , i.e. “equality is transitive”.*

Proof. The type we are looking to build an element of is `forall (A : Type) (x y z : A), x = y -> y = z -> x = z`.

This seems like a more cumbersome task, and indeed if we were to write the element explicitly it would be quite a long expression. However, that does not mean that writing a formalized proof in Coq has to be awkward or tedious – in fact it can even be easier than attempting to write a comprehensive proof on paper. This is due to the existence of *tactics*, which are commands that can be used to build an element of a type in a more natural proof style, while maintaining correctness.

To initiate a proof by tactics, instead of writing `:=` after the type of the element being defined, we write a full stop, followed by the word **Proof**. After reading this, Coq will give us a *goal*, which is simply the type we are trying to build an element of.

Then, we write tactics to change our goal into more manageable ones. Since the goal is a dependent function, we can use the tactic **intros** to introduce the arguments of the function. This tactic changes the goal from constructing a dependent function to constructing an element of its co-domain. Thus, after running the command

```
intros A x y z p q.
```

, we have in our context the hypotheses

```
A : Type
x, y, z : A
p : x = y
q : y = z
```

and the goal has changed to

```
x = z
```

Now, we want to perform induction on p , i.e., suppose $y \equiv x$ and $p \equiv \text{refl}_x$. This is done with the tactic **induction**. This tactic always takes a hypothesis as an argument, and applies the appropriate induction principle to it. After running the command

```
induction p
```

our new hypotheses are

```
A : Type
x, z : A
q : x = z
```

and the new goal is

```
x = z
```

Now, if we write **induction q**, the hypotheses change to

```
A : Type
x : A
```

and the goal to

```
x = x
```

By this point, we have a proof of the goal, i.e. an element of type $x = x$, namely refl_x . We can use the `exact` tactic to provide it and complete the goal (writing `exact (reflexivity x)`), or, more idiomatically, we can use the tactic `reflexivity`, which closes goals that look like $X = X$. The complete definition is therefore

```
Definition conc:
forall {A : Type} {x y z : A}, x = y -> y = z -> x = z.
Proof.
intros A x y z p q.
induction p.
induction q.
reflexivity.
Defined.
```

□

In practice, we will write this concatenation of paths with infix notation, i.e. as $p \bullet q$. This is also the case in the Coq-HoTT library, where we can write $p @ q$. We will of course right p^{-1} for $\text{inv } p$ ($p^\wedge$ in the library). Since we used path induction to define them, concatenation of paths comes with the computation rule $\text{refl}_x \bullet \text{refl}_x \equiv \text{refl}_x$, and inverses with $\text{refl}_x^{-1} \equiv \text{refl}_x$.

In the next section, we will explore more properties of these operations, as well as how functions and type families interact with equality.

4. TYPES AS GROUPOIDS

Concatenation and inverses suggest a groupoid structure on paths, with refl_x as identities. But how can we prove that they obey the laws of a groupoid, for example that concatenation is associative or that inverses cancel? The tools described so far give us a way to represent that internally, as *paths between paths*, i.e. as equalities over a type which is itself an equality type. Of course then we can ask about the properties of those “two-dimensional” paths, and so on, and end up with an ∞ -groupoid structure. However, an advantage of type theory is that we never have to go through the complicated combinatorics of describing this structure explicitly: everything is already determined by path induction, and as such we may uncover only the structure that we need for the results we are interested in.

4.1. Higher paths and operations. Higher paths include unit laws, associativity, and cancellation. If $A : \mathcal{U}$, $x, y, z, w : A$, we have:

- Unit laws:
 - $\text{concat_lp} : \prod_{p:x=y} \text{refl}_x \bullet p = p$
 - $\text{concat_p1} : \prod_{p:x=y} p \bullet \text{refl}_x = p$
- Associativity: $\text{concat_assoc} : \prod_{p:x=y} \prod_{q:y=z} \prod_{r:z=w} (p \bullet q) \bullet r = p \bullet (q \bullet r)$
- Cancellation laws:
 - $\text{concat_Vp} : \prod_{p:x=y} p^{-1} \bullet p = \text{refl}_y$
 - $\text{concat_pV} : \prod_{p:x=y} p \bullet p^{-1} = \text{refl}_x$
- Whiskering:
 - $\text{whiskerL} : \prod_{p,q:y=z} \prod_{r:x=y} (p = q) \rightarrow (r \bullet p) = (r \bullet q)$
 - $\text{whiskerR} : \prod_{p,q:x=y} \prod_{r:y=z} (p = q) \rightarrow (p \bullet r) = (q \bullet r)$

4.2. Applying a function to a path. One feature of the equality we've introduced is that it is respected by functions, i.e., if we have a function $f : A \rightarrow B$ and a path $p : a =_A b$, we have a path $\mathbf{ap}_f p : (fa) = (fb)$. Moreover, this construction is “natural”, i.e., we have an element

$$(1) \quad \mathbf{ap} : \prod_{A,B:\mathcal{U}} \prod_{f:A \rightarrow B} \prod_{a,b:A} ((a = b) \rightarrow (fa = fb))$$

We shall usual leave the arguments A , B , a and b implicit, as they are deducible from p , and write the argument f in subscript (as above).

The way to construct $\mathbf{ap}$ is through path induction. Let us only present the proof only in Coq, as this proof follows the exact pattern of the previous : apply path induction and use reflexivity.

Definition `ap {A B : Type} (f : A -> B) {a b : A}: (a = b) -> ((f a) = (f b)).`

Proof.

```
intro p.
induction p.
reflexivity.
```

Defined.

The function $\mathbf{ap}_f : \prod_{a,b:A} (a = b) \rightarrow ((fa) = (fb))$, can be though of as “ f at the level of morphisms”. And of course identity types are types in their own right, so we can iterate $\mathbf{ap}$ to get f at any level. This, along with some coherence laws, which themselves are provable by induction as well, means that every function between types is automatically a *functor* between ∞ -groupoids. Notice that the computation rule for `ind=` gives us the judgemental equality $\mathbf{ap}_f \mathbf{refl}_a \equiv \mathbf{refl}_{fa}$, which can be read as “ f respects identities”.

4.3. Transporting along a path. A special case of this is when f is a type family. That is, if we have $P : A \rightarrow \mathcal{U}$, and $a =_A b$, what can we say about $P(a)$ and $P(b)$? The discussion above tells us that these two types are equal, but that does not give us (without any further effort) any way to translate between elements of $P(a)$ and elements of $P(b)$. That is where **transport** comes in. More specifically, we have an element

$$(2) \quad \mathbf{transport} : \prod_{A:\mathcal{U}} \prod_{P:A \rightarrow \mathcal{U}} \prod_{a,b:A} \prod_{p:a=b} P(a) \rightarrow P(b)$$

, which is obtained by performing induction on p and extracting the identity function. We will leave unnecessary arguments implicit, and write the type family in superscript.

Naturally, transporting along p^{-1} gives a function in the other direction, and it is also a direct application of path-induction that for all $x : A$, $\mathbf{transport}^P p^{-1} (\mathbf{transport}^P p x) = x$ and for all $y : B$, $\mathbf{transport}^P p (\mathbf{transport}^P p^{-1} y) = y$, i.e. $\mathbf{transport}^P p$ and $\mathbf{transport}^P p^{-1}$ are homotopical inverses.

4.4. Applying a dependent function to a path. Above, we only defined the action on paths of *non-dependent* functions. If we attempt to define it for dependent functions, we run into a problem: for $f : \prod_{x:A} P x$, and $p : a =_A b$, the type $fa = fb$ might not even make sense, since Pa needn't be *judgementally* Pb . Thankfully, **transport** answers that problem: by transporting fa along the path p , we get an element in Pb , which we may then compare with $fb : Pb$. This way, we have the following definition.

Definition `apd {A : Type} {P : A -> Type} (f : forall (x : A), P x) {a b : A} (p : a = b):`
`transport P p (f a) = (f b).`

Proof.

```
induction p.
```

reflexivity.

Defined.

Thus, apd_f maps a path $p : a =_A b$ to a path in $\text{transport}^P p (f a) =_{P b} (f b)$. We will write statements of this kind fairly often, so we will introduce the notation $f a =_p^P f b$ with that meaning (in Coq we'll write $p \# (f a) = (f b)$). The type family P is not specified as it can be inferred from $f a$ and $f b$.

Since a (non-dependent) function can be seen as a dependent function over a constant type family, one could ask how ap and apd are related in this case. To answer that, we first check how transport behaves over constant type families. Well, we have $\text{transport}^{\lambda x. B} \text{refl}_a x \equiv x$, so by induction we should have $\text{transport}^{\lambda x. B} p x = x$. This is indeed the case, and we can define it in the following way.

Definition `transport_const` {A : Type} (B : Type) {a b : A}:

forall (p : a = b) (x : B), transport (fun _ => B) p x = x.

Proof.

induction p.

reflexivity.

Defined.

Now, $\text{apd}_f p$ is a path between $\text{transport}^{\lambda x. B} (f a)$ and $(f b)$, and we have a path $\text{transport_const } B p (f a)$ from $\text{transport}^{\lambda x. B} (f a)$ to $f a$, while ap_f is a path from $(f a)$ to $(f b)$. If we concatenate the latter two, we obtain something comparable to apd_f .

Definition `apd_const` {A B : Type} (f : A -> B) {a b : A} :

forall (p : a = b), apd f p = transport_const B p (f a) @ ap f p.

Proof.

induction p.

reflexivity.

Defined.

These elements will come into play when dealing with higher inductive types.

4.5. Homotopies. Since functions are automatically “functors”, one could ask what objects play the role of natural transformations. In category theory, a natural transformation η between two functors $F, G : \mathcal{C} \rightarrow \mathcal{D}$ corresponds to an arrow $\eta_c : F(c) \rightarrow G(c)$ for every object in $\mathcal{C}$ (where this collection of arrows satisfies a naturality condition). In type theory, the direct translation of this is the type $\prod_{x:A} f x = g x$, if $f, g : A \rightarrow B$ ⁶. We call this the type of *homotopies* between f and g . It is standard in the Coq-HoTT library to use the notation $f == g$ to mean this type, and in this text we will write it as $f \sim g$.

In keeping with the theme of path induction making every construction natural, we can check that homotopies satisfy the same condition as natural transformations do in category theory, i.e. for types A and B , functions $f, g : A \rightarrow B$, a homotopy $H : \prod_{x:A} f x = g x$, and a path $p : a =_A b$, we have that the following square (of equalities) commutes.

$$\begin{array}{ccc} f a & \xrightarrow{\text{ap}_f p} & f b \\ H a \downarrow & & \downarrow H b \\ g a & \xrightarrow{\text{ap}_g p} & g b \end{array}$$

i.e., we have the statement

⁶ f and g can just as well be dependent functions over the same type family, and this notion still makes sense

Definition concat_ap_hmt {A B : Type} (f g : A -> B) (H : f == g) {a b : A} (p : a = b) :
 ap f p @ H b = H a @ ap g p.

Proof.

induction p.

exact ((concat_1p (H a)) @ (concat_p1 (H a)))~).

Defined.

This proof is an example of a powerful idea in Homotopy Type Theory: in order to prove something about a particular path, we first prove a lemma about *all* paths and then apply it to the path in question. After executing the command `induction p`, we are left with the goal $\text{refl}_a \cdot H a = H a \cdot \text{refl}_a$. These two sides are not judgementally equal, and there are no more paths with free endpoints that we can apply induction to. However, we can instantiate the lemmas `concat_1p` and `concat_p1` that we've already proved by induction.

4.6. Equivalences. Now that we have an analogous notion of natural transformation, we can define what an equivalence between types is. If we have two types A and B and functions $f : A \rightarrow B$ and $g : B \rightarrow A$, we say that f and g are *quasi-inverses* if we have homotopies $\alpha : f \circ g \sim \text{id}_B$ and $\beta : g \circ f \sim \text{id}_A$. In this case, we can say that A and B are equivalent types.

However, defining the type of equivalences between A and B is a more intricate problem, which has several possible answers. Given a function $f : A \rightarrow B$, we could define the type of witnesses that f is an equivalence as $\sum_{g:B \rightarrow A} (f \circ g \sim \text{id}_B) \times (g \circ f \sim \text{id}_A)$. However, this is not the most convenient way. As is the case in category theory, we can come up with a notion of an *adjoint* equivalence. We can call the type of witnesses that f is an adjoint equivalence `lsEquiv f`. This has the following nice properties:

- we have a function $\prod_{f:A \rightarrow B} \text{lsEquiv } f \rightarrow \sum_{g:B \rightarrow A} (f \circ g \sim \text{id}_B) \times (g \circ f \sim \text{id}_A)$;
- as well as a function $\prod_{f:A \rightarrow B} \sum_{g:B \rightarrow A} (f \circ g \sim \text{id}_B) \times (g \circ f \sim \text{id}_A) \rightarrow \text{lsEquiv } f$.
- given $f : A \rightarrow B$, any two witnesses that f is an adjoint equivalence are equal.

I.e., if f has a quasi inverse, we can modify it to produce a canonical, adjoint equivalence. We define the type $A \simeq B$ (`A <~> B` in Coq) to be the type $\sum_{f:A \rightarrow B} \text{lsEquiv } f$, i.e, the functions $A \rightarrow B$ that are equivalences. We won't concern ourselves with the details of these, which can be found in [5, Chapter 4]. It is enough to know that the function

$$\prod_{AB:\mathcal{U}} \prod_{f:A \rightarrow B} \left(\sum_{g:B \rightarrow A} (f \circ g \sim \text{id}_B) \times (g \circ f \sim \text{id}_A) \right) \rightarrow \text{lsEquiv } f$$

, is defined in the Coq-HoTT `isequiv_adjointify`. Thus, to construct an element of $A \simeq B$ we need only provide quasi-inverses.

Even though an equivalence is technically a pair $e \equiv (e_0, e_1) : \sum_{f:A \rightarrow B} \text{lsEquiv } f$, we will often say “the equivalence e ” when we really mean “the function e_0 ” and vice-versa. This is not harmful, since it can be inferred from the context which one we are talking about – for example, if we apply e to an element of A we must mean the function. This also works in the context of the Coq-HoTT library: i.e. we can apply an equivalence to an element and Coq will apply the underlying function, because that is the only interpretation that makes sense.

4.7. Naturality Results. There are many more interesting naturality results relating to `transport` and `ap` that follow from path induction (and instantiating previously proved lemmas). Here, we state in the Coq syntax those that will be useful later on, accompanied by some hand-drawn pictures for geometric intuition.

Definition transport_postcompose {A B : Type} (P : B -> Type) (f : A -> B) {x y : A} (p : x = y) :

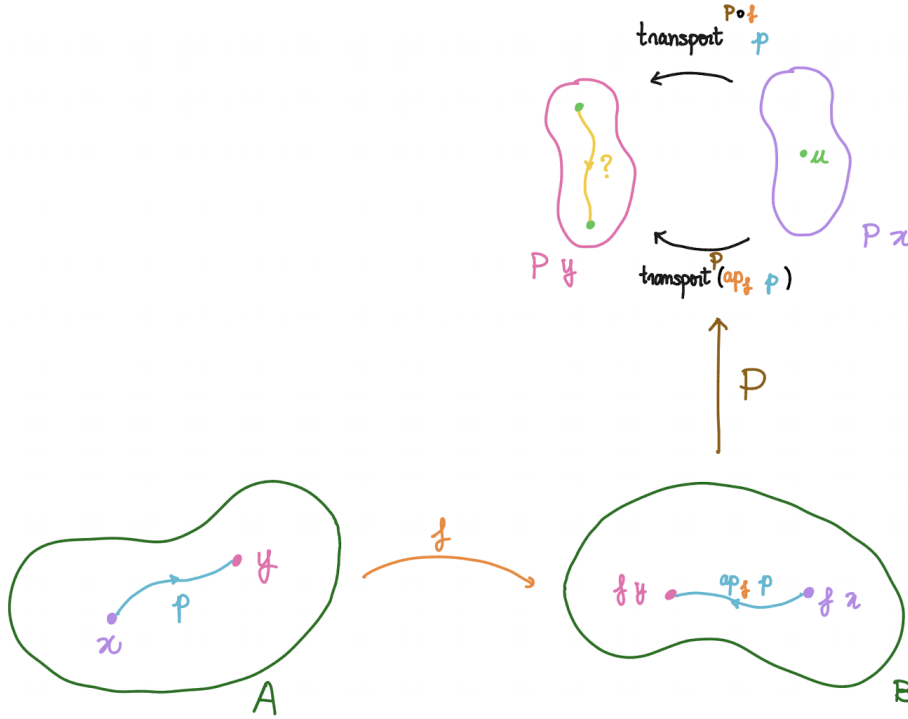


FIGURE 1. visualization of transport_posctompose

```
transport (P o f) p == transport P (ap f p).
```

Proof.

```
  induction p.
```

```
  reflexivity.
```

Defined.

```
Definition transport_paths_FlFr (A B : Type) (f g : A -> B) {x y : A} (p : x = y):
forall (q : f x = g x), transport (fun z => f z = g z) p q = (ap f p)^ @ q @ (ap g p).
```

Proof.

```
  induction p.
```

```
  intro q.
```

```
  refine (_ @ (whiskerR (concat_1p q) (reflexivity (g x))))^).
```

```
  exact ((concat_p1 q)^).
```

Defined.

Note the use of the tactic `refine`. This tactic matches the goal against the co-domain of a function, in this case $\lambda p. p \cdot (\text{whiskerR}(\text{concat_1p} q) \text{refl}_{gx})^{-1}$, and turns the goal into the missing argument(s) of the function, written with “`_`”. This effectively allows us to work backwards from the goal.

`transport_paths_FlFr` has several special cases : for example when $B \equiv A$, $f \equiv \text{id}_A$ and $g \equiv \lambda z. a$ for some $a : A$ it reduces to $\text{transport}^{\lambda z. z = a} p q = p^{-1} \cdot q$. We call this case `transport_paths_r`.

```
Definition transport_arrow {X : Type} (A B : X -> Type) {x y : X}
```

```
(p : x = y) (f : A x -> B x):
```

```
  p # f = (fun u => transport B p (f (transport A p^ u))).
```

Proof.

```
  induction p.
```

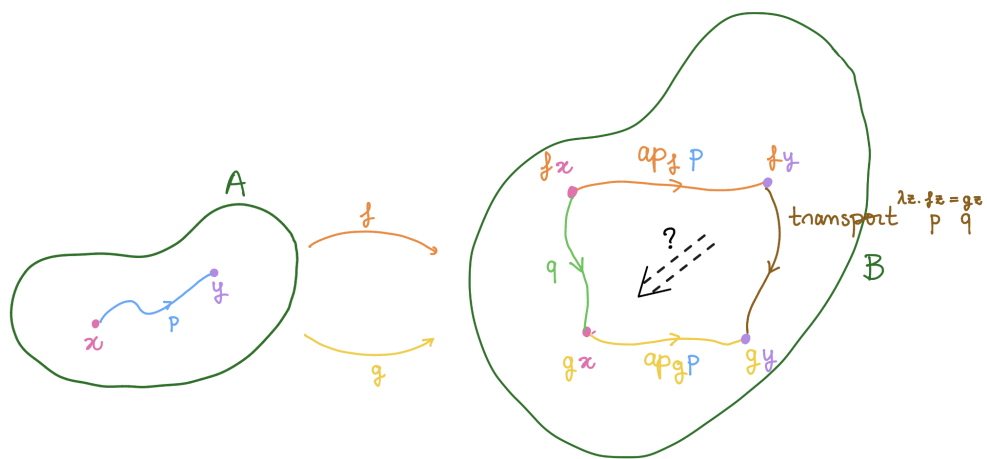


FIGURE 2. visualization of transport_paths_FIFr

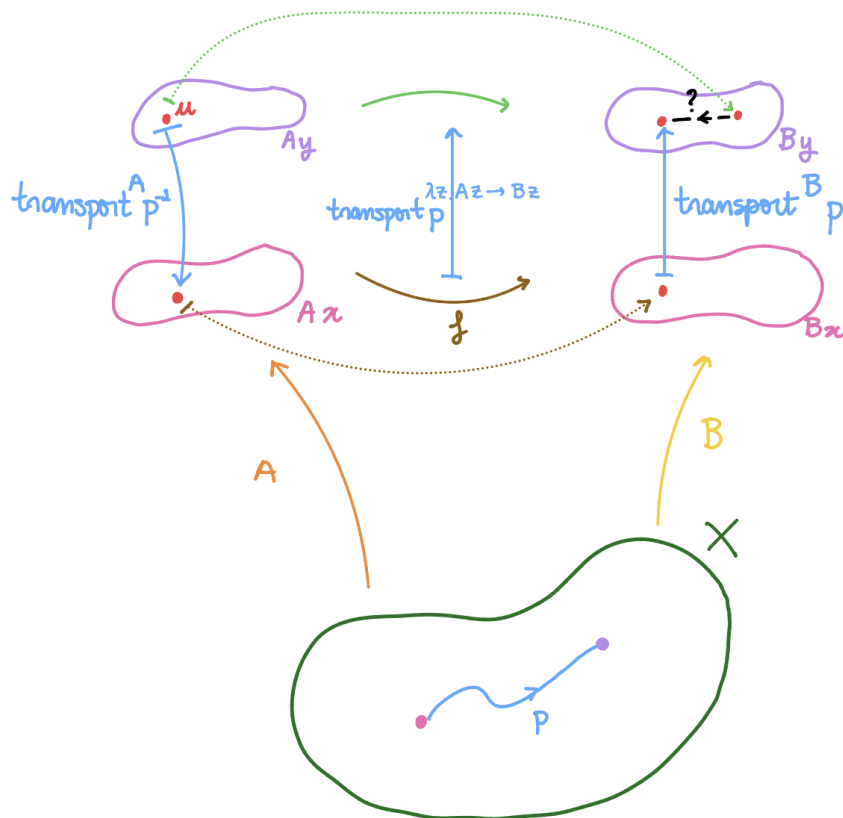


FIGURE 3. visualization of transport_arrow

reflexivity.
Defined.

5. CHARACTERIZING EQUALITY IN TYPE FORMERS

5.1. Equality in Function Types. We can easily turn equalities between (dependent) functions into homotopies in a functorial way, using the following element:

Definition `happly` $\{A : \text{Type}\} \{P : A \rightarrow \text{Type}\} (f\ g : \text{forall } (x : A), P\ x) :$
 $f = g \rightarrow f == g.$

Proof.

```
intro p.
induction p.
exact (fun x => reflexivity (f x)).
```

Defined.

We would like to have a converse, i.e., a way to turn homotopies into equalities that makes `happly` an equivalence. However, there is nothing we can use to assemble a collection of paths $\prod_{x:A} f\ x = g\ x$ into a “global” path $f = g$. Thus, we introduce it as an axiom: the axiom of function extensionality.

Axiom 5.1.1.

$$\text{funext} : \prod_{A:\mathcal{U}} \prod_{P:A \rightarrow \mathcal{U}} \prod_{fg:\prod_{x:A} P\ x} \text{IsEquiv}(\text{happly } f\ g)$$

From this we can extract a function $f \sim g \rightarrow f = g$. This function is implemented in the Coq-HoTT library as `path_arrow`.

We can also extract the homotopies between the two compositions and respective identities, and from these conclude that the operations on paths between functions are performed point-wise [5, Section 2.9].

5.2. Equality in Universes. Just as an equality between functions gives rise to a homotopy, an equality between types induces an equivalence. If we have a path $p : A \simeq B$, then we can extract functions $f : A \rightarrow B$ and $g : B \rightarrow A$ by transporting along p and p^{-1} , over the type family $\text{id}_{\mathcal{U}}$. These are quasi-inverses, as stated in subsection 4.3. Formally, we have:

Definition `equiv_path` $\{A\ B : \text{Type}\}:$

$A = B \rightarrow A <\sim> B.$

Proof.

```
intro p.
refine (Build_Equiv A B (transport idmap p) _).
refine (isequiv_adjointify (transport idmap p) (transport idmap p^_) _ _).
* induction p.
  reflexivity.
* induction p.
  reflexivity.
```

Defined.

As was the case for function types, to provide a converse we need to introduce an extra axiom: the univalence axiom.

Axiom 5.2.1.

$$\text{ua} : \prod_{A,B:\mathcal{U}} \text{IsEquiv}(\text{equiv_path } A\ B)$$

The function associated to this equivalence is written in the library as `path_universe`. The homotopies in this equivalence can be written as:

- $_ : \text{transport}^{\text{id}_{\mathcal{U}}}(\text{ua } f) \sim f$
- $_ : \prod_{p:A=B} \text{ua}(\text{transport}^{\text{id}_{\mathcal{U}}} p) = p$

The first one acts as a propositional computation rule for the univalence axiom, and is written `transport_idmap_path_universe` in the library, while the second acts as a propositional uniqueness principle.

5.3. Equality in Coproducts. We choose to characterize equality in co-product types for two reasons. First, this will allow us to prove our first *negative result*: that $\text{inl } a \neq \text{inr } b$ ⁷, which an essential step to show that univalence introduces non-trivial paths. The second reason is that this proof is a good introduction to the *encode-decode method* – a typical tool in Homotopy Type Theory – which we will adopt to calculate the fundamental group of the circle.

Our aim is to prove the following theorem.

Theorem 5.3.1. *Let $A, B : \mathcal{U}$, $a_0, a_1 : A$ and $b_0, b_1 : B$. Then:*

- $\text{inl } a_0 =_{A+B} \text{inl } a_1 \simeq a_0 =_A a_1$
- $\text{inl } a_0 =_{A+B} \text{inr } b_0 \simeq \mathbf{0}$
- $\text{inr } b_0 =_{A+B} \text{inr } b_1 \simeq b_0 =_B b_1$
- $\text{inr } b_0 =_{A+B} \text{inl } a_0 \simeq \mathbf{0}$

We will focus on the former two items, since the latter two are analogous to these. We are proving something about equality, so we would like to use path induction. However, it is not possible to do so when, as in this case, both ends of the equality are fixed. The encode-decode method consists of letting one of the endpoints of the path we are interested vary, and proving something *more general* than we want, but that is *easier to prove*, since we unlock the power of path induction.

With that in mind, we will define the type family `code` by coproduct induction, in the following way.

Definition code

```
{A B : Type} (a0: A) : A + B -> Type.
```

Proof.

```
intro x.
induction x as [a | b].
* exact (a0 = a).
* exact Empty.
```

Defined.

, i.e., `code  $a_0$  (inl  $a$ )` $\equiv (a_0 = a)$, while `code  $a_0$  (inl  $b$ )` $\equiv \mathbf{0}$. In other words, `code  $a_0$   $x$` is exactly what we *want* the type $a_0 =_{A+B} x$ to be. Our goal is to prove the statement

$$(3) \quad \prod_{x:A+B} \prod_{a_0:A} \prod_{x:A+B} (a_0 =_{A+B} x) \simeq (\text{code } a_0 x)$$

We will construct quasi-inverses. From left to right, we need a way to produce an element of `code  $a_0$   $x$` from $p : a_0 = x$. Transport along p over `code  $a_0$  :  $A + B \rightarrow \mathcal{U}$` gives us a function `code  $a_0$   $a_0 \rightarrow$  code  $a_0$   $x$` , so we just need an element of `code  $a_0$   $a_0 \equiv a_0 = a_0$` , which can be refl_{a_0} . Thus, `encode`, is defined as follows.

Definition encode {A B : Type} (a0 : A) : forall (x : A + B), (inl B a0) = x -> code a0 x.

Proof.

```
intros x p.
exact (transport (code a0) p (reflexivity a0)).
```

Defined.

⁷ $x \neq y$ is notation for $x = y \rightarrow \mathbf{0}$

For the other direction, we want to produce a dependent function $\prod_{x:A+B}(\text{code } x) \rightarrow (\text{inl } a_0 = x)$. We may do this by coproduct induction. When x is $\text{inl } a$, $\text{code } a_0 x \equiv a_0 = a$, and there is an obvious element of $(a_0 = a) \rightarrow (\text{inl } a_0 = \text{inl } a)$, namely ap_{inl} . When x is $\text{inr } b$ we have a contradiction, $\text{code } \text{inr } b \equiv 0$. Thus, decode is defined as follows.

```
Definition decode {A B : Type} (a0 : A) :
forall (x : A + B), code a0 x -> (inl B a0) = x.
```

Proof.

```
  induction x.
  * exact (ap (fun c => inl B c)).
  * contradiction.
```

Defined.

To complete the proof, we need homotopies $\text{encode_decode} : (\text{encode } a_0 x) \circ (\text{decode } a_0 x) \sim \text{id}_{\text{code } a_0 x}$ and $\text{decode_encode} : (\text{decode } a_0 x) \circ (\text{encode } a_0 x) \sim \text{id}_{\text{inl } a_0 = x}$ for each $a_0 : A$ and $x : A + B$. Let $p : \text{inl } a_0 =_{A+B} x$.

For $\text{decode_encode } a_0 x$, let $p : \text{inl } a_0 = x$. This is where the great advantage of this method comes in: p is a path in $A + B$ between the fixed $\text{inl } a_0$ and the *variable* x , so we may use path induction and assume $x \equiv \text{inl } a_0$ and $p \equiv \text{refl}_{\text{inl } a_0}$. The goal then becomes solvable by reflexivity.

```
Definition decode_encode {A B : Type} (a0 : A) (x : A + B):
(decode a0 x) o (encode a0 x) == idmap.
```

Proof.

```
  intro p.
  induction p.
  reflexivity.
```

Defined.

For the other homotopy, let $c : \text{code } a_0 x$. We want to show that $(\text{encode } a_0 x)(\text{decode } a_0 x) = c$. We use induction on x to split the goal into the cases $x \equiv \text{inl } a$ and $x \equiv \text{inr } b$.

In the first case, $\text{decode } a_0 x \equiv ap_{\text{inl}}$, so the goal becomes $\text{transport}^{\text{code } a_0} (ap_{\text{inl}} c) \text{refl}_{a_0} = c$. We may use path induction on $c : a_0 =_A a$ to complete the goal immediately. In the second case, we have $c : \text{code } a_0 \text{inr } b \equiv 0$, a contradiction.

```
Definition encode_decode {A B : Type} (a0 : A) (x : A + B):
(encode a0 x) o (decode a0 x) == idmap.
```

Proof.

```
  intro c.
  induction x.
  * induction c.
    reflexivity.
  * contradiction.
```

Defined.

We now have all the ingredients necessary to give the family of equivalences in 3. The important particular case we specifically wanted is

$$\prod_{A,B:\mathcal{U}} \prod_{a:A} \prod_{b:B} (\text{inl } a \neq \text{inr } b)$$

We can supply the element $\text{inl_neq_inr} \equiv \lambda A B a b. \text{encode } a (\text{inr } b)$ of this type, and even more specifically $\text{zero_neq_one} \equiv \text{inl_neq_inr } 1 \ 1 \ \star \ \star : 0_2 \neq 1_2$.

6. SOME CONSEQUENCES OF UNIVALENCE

Univalence is a powerful tool in proving negative results, i.e., of constructing functions into the empty type. One such result is the existence of a non-trivial loop. The other consequence we are going to explore exposes a “flaw” in the propositions-as-types correspondence.

6.1. Sets and Propositions. Before proving these results, it will be useful to introduce some terminology. We have used language like “if we view this type as a proposition”. What that means is that we don’t care about the particular elements of the type, just whether it is inhabited. This is particularly valid if all the elements of that type are equal, which motivates the following definition:

Definition 6.1.1. We say $A : \mathcal{U}$ is a mere proposition if, for every $x, y : A$, $x =_A y$. We define the type family

$$\text{IsProp} \equiv \lambda(A : \mathcal{U}). \prod_{x, y : A} (x = y) : \mathcal{U} \rightarrow \mathcal{U}$$

, the proofs that A is a mere proposition.

Similarly, we can define a set as a type whose equality types are mere propositions.

Definition 6.1.2. We define the type family

$$\text{IsSet} : \lambda(A : \mathcal{U}). \prod_{x, y : A} \text{IsProp}(x = y) : \mathcal{U} \rightarrow \mathcal{U}$$

Examples of mere propositions are $\mathbf{0}$, $\mathbf{1}$, $\text{IsProp } A$, $\text{IsSet } A$ for any type A [5, Section 3.1], and $\text{IsEquiv } f$ for any function f [5, Chapter 4]. It is an intuitive result that any mere proposition is either equivalent to $\mathbf{1}$ (if it’s inhabited) or $\mathbf{0}$ (if it’s not). In the first case we call the proposition *contractible*. Examples of sets (that are not propositions) are $\mathbb{N}$ and $\mathbf{2}$ [5, Chapter 3].

In this language, the first result we are going to prove shows that not every type is a set, while the second will show that we need to be careful when interpreting types that are not mere propositions as such.

6.2. A non-trivial loop. We want to use univalence to prove a statement of the form $(p =_A \text{refl}_x) \rightarrow \mathbf{0}$. One good idea is to pick A to be a type universe, x to be a type and p to be the result of applying the univalence axiom to some non-trivial autoequivalence. Thus, we want to choose x as simple as possible, but with enough data so as to allow for a non-identity endomorphism – the Booleans answer that description. Thus, we define $e : \mathbf{2} \rightarrow \mathbf{2}$ by induction as $e \mathbf{0}_2 \equiv \mathbf{1}_2$ and $e \mathbf{1}_2 \equiv \mathbf{0}_2$.

The first step is to check that e is an equivalence. But e is its own quasi-inverse, since $(e \circ e) \mathbf{0}_2 \equiv \mathbf{0}_2$ and $(e \circ e) \mathbf{1}_2 \equiv \mathbf{1}_2$, which, by induction, are the only cases we need to verify.

This way, we have $p \equiv \text{ua } e : \mathbf{2} = \mathbf{2}$. To give a function in $p = \text{refl}_2 \rightarrow \mathbf{0}$, we take an arbitrary proof of $p = \text{refl}_2$, say c , and construct a contradiction. We will do this by repeatedly using the tactic `pose`, which creates local abbreviations for elements we define.

Definition `p_neq_refl : (p = reflexivity Bool) -> Empty.`

Proof.

```
intro c.
pose (r1 := ap (equiv_path Bool Bool) c).
pose (r2 := (equiv_path_path_universe E)^ @ r1).
pose (r3 := ap equiv_fun r2).
pose (r4 := apD10 r3 (inr tt)).
exact (zero_neq_one r4).
```

Defined.

This proof can look quite unreadable. However, as we write it in Coq, it shows us the types of the elements we are introducing, which makes it just like writing the following on paper:

$$\begin{array}{ll}
 p = \text{refl}_2 & \text{by hypotheses} \\
 \text{equiv_path } p = \text{id}_2 & \text{by ap(equiv_path 2 2)} \\
 e = \text{id}_2 & \text{by (equiv_path_path_universe e)}^{-1} \\
 \prod_{x:2} (e\ x = x) & \text{by happly} \\
 0_2 = 1_2 & \text{by application at } 1_2 \\
 0 & \text{by zero_neq_one}
 \end{array}$$

It is worthwhile to take a moment to think about what we've just proved. We already had a preexisting negative result, namely that $0_2 \neq 1_2$. Using univalence, we were able to turn this result about points into a negative result about *paths*: we proved that $\mathcal{U}_0$, has a richer path structure that makes it not a set. In the next sections we will be able to exploit this to prove that the circle is also not set.

6.3. Double negation. Another negative result we can prove using this non-trivial loop is that the law of double negation does not hold for all types. More precisely, we have the following theorem.

Theorem 6.3.1. *We have an element of the type*

$$\neg \prod_{A:\mathcal{U}} ((\neg\neg A) \rightarrow A)$$

The proof is quite technical. Clearly, the thing to do is to consider A to be the Booleans. However, we won't prove the stronger statement $\neg((\neg\neg 2) \rightarrow 2)$. This is because we need a dependent function $f : \prod_{A:\mathcal{U}} ((\neg\neg A) \rightarrow A)$, to which we can apply constructions like `apd`, in order to derive a contradiction, and not just an element of $((\neg\neg 2) \rightarrow 2)$.

The first observation that will help us prove theorem 6.3.1 is that for any type B , $\neg B$ is a mere proposition. This is a consequence of function extensionality.

Definition `neg_is_prop` {A : Type} (u v : ~ A) : u = v.

Proof.

```

  apply (path_arrow).
  intro f.
  contradiction.

```

Defined.

Now, this is where we can spot a difference between $\neg\neg 2$ and 2 : the former is a mere proposition, while the latter is not, due to `zero_neq_one`. This doesn't mean, however, that we can't find a map $\neg\neg 2 \rightarrow 2$ – indeed we can, since $\neg\neg 2 \simeq 1$ and a map $1 \rightarrow 2$ is simply a choice of an element of 2 , of which there are two – we just can't do it in a *functorial* way, i.e., one that would extend to a map $\prod_{A:\mathcal{U}} \neg\neg A \rightarrow A$.

To prove this, suppose we have $f : \prod_{A:\mathcal{U}} \neg\neg A \rightarrow A$ (and write the argument A in subscript). We will first prove the following intermediate statement.

Lemma 6.3.2. *For all $u : \neg\neg 2$, we have $f_2 u = e(f_2(\text{transport}^{\neg\neg} p^{-1} u))$, where $\neg\neg$ stands for the type family $\lambda X. \neg\neg X : \mathcal{U} \rightarrow \mathcal{U}$ and $p : 2 = 2$ is the non-trivial path defined before.*

Proof. The idea is that we can combine our two main ingredients, $f : \prod_{A:\mathcal{U}} \neg\neg A \rightarrow A$ and $p : \mathbf{2} =_{\mathcal{U}} \mathbf{2}$ using `apd`, resulting in:

$$\text{apd } f \, p : \text{transport}^{\lambda X. \neg\neg X \rightarrow X} p \, f_{\mathbf{2}} = f_{\mathbf{2}}$$

From here, we just apply lemmas previously derived, as well as the computation rule for univalence. The formalized proof is written below.

Definition nn := fun X => $\neg\neg X$.

Definition f2_precomposed (f : forall (A : Type), ($\neg\neg A$) -> A) (u : $\neg\neg \text{Bool}$) : f Bool u = e0 (f Bool (transport nn p^u)).

Proof.

```
pose (q := apd f p).
pose (r := qu @ transport_arrow nn idmap p (f Bool)).
pose (s := happly (transport_idmap_path_universe E) (f Bool (transport nn pu))).
exact (happly r u @ s).
```

Defined.

□

What we proved in this lemma was a homotopy between $f_{\mathbf{2}}$, and $e \circ f_{\mathbf{2}}$, precomposed with an endomorphism of $\neg\neg\mathbf{2}$, namely $\text{transport}^{\neg\neg} p^{-1}$. But as per our previous discussion $\neg\neg\mathbf{2}$ is a mere proposition, so any endomorphism of it must be homotopic to the identity, and, by naturality, we can ignore it. Thus we obtain a homotopy

$$\prod_{u:\neg\neg\mathbf{2}} f_{\mathbf{2}} u = e(f_{\mathbf{2}} u)$$

, which is formally proved as follows.

Definition contr_hmt (f : forall (A : Type), ($\neg\neg A$) -> A) : f Bool == E o f Bool.

Proof.

```
intro u.
refine (f2_precomposed f u @ _).
refine (ap (E o (f Bool)) _).
exact (neg_is_prop (transport nn pu u)).
```

Defined.

This homotopy gives us a fixed point of e for each element of $\neg\neg\mathbf{2}$. In order to find a contradiction, all we need to do is prove that e has no fixed points, and find an element of $\neg\neg\mathbf{2}$.

Proving that e has no fixed points is easily done by induction on the booleans and using `zero_neq_one` and its inverse.

Definition no_fixed_points: forall (b : Bool), $\neg(b = E \, b)$.

Proof.

```
induction b.
* induction a.
  exact zero_neq_one.
* induction b.
  exact (fun p => zero_neq_one pu).
```

Defined.

An element of $\neg\neg\mathbf{2}$ is a function that maps functions in $\mathbf{2} \rightarrow \mathbf{0}$ to elements of $\mathbf{0}$. Of these we can produce two: evaluation at $0_{\mathbf{2}}$ and evaluation at $1_{\mathbf{2}}$. Choose $u \equiv \lambda g. g \, 0_{\mathbf{2}} : \neg\neg\mathbf{2}$. Thus, we have our desired result:

Definition no_double_neg: $\sim(\text{forall } (A : \text{Type}), (\sim\sim A) \rightarrow A)$.

Proof.

```
intro f.
refine (no_fixed_points (f Bool _) (contr_hmt f _)).
exact (fun g => g zero2).
```

Defined.

7. HIGHER INDUCTIVE TYPES

Inductive types already give our type theory a lot of expressive power, and allow for some interesting consequences of Univalence. However, we are not yet able to define some concrete non-trivial topological spaces that we'd wish to see realised: in fact, all the concrete types we've defined that don't involve universes are sets, and as such have no interesting topological properties like non-trivial loop spaces.

In higher inductive types, we allow not only point (i.e., zero-dimensional) constructors, but also for path constructors of any dimension. The induction principles require us to specify the images of functions on paths, and will give us propositional β -conversion rules for these, which will have to be introduced as extra axioms for every type we define.

There are many constructions that are made possible, such as co-equalizers of functions (and hence all co-limits), set quotients, CW-complexes, etc. [5, Chapter 6]. Here, we will focus on the two simplest, which already demonstrate some of the power of higher inductive types : the interval and the circle.

7.1. The Interval. The interval I is defined as follows.

Definition 7.1.1. We have a higher inductive type, I , with the following constructors.

- $0_I : I$
- $1_I : I$
- $\text{seg} : 0_I = 1_I$

Its induction principle is

$$\text{ind}_I : \prod_{P:I \rightarrow \mathcal{U}} \prod_{b_0:P} \prod_{b_1:P} (b_0 =_{\text{seg}}^P b_1) \rightarrow \prod_{x:I} P x$$

It comes with the judgemental computation rules

- $\text{ind}_I P b_0 b_1 s 0_I \equiv b_0$
- $\text{ind}_I P b_0 b_1 s 1_I \equiv b_1$

Along with the axiom

- $_ : \text{apd}_{\text{ind}_I P b_0 b_1 s} = s$

From this induction principle, we can derive a non-dependent version (i.e., a *recursion* principle) by choosing P to be a constant type family $\lambda x. C$ and applying the propositional “computation rule” $\text{transport}_{_ \text{const } C} \text{seg } b_0 : \text{transport}^{\lambda x. C} \text{seg } b_0 = b_0$, to get

$$\text{rec}_I : \prod_{C:\mathcal{U}} \prod_{b_0:C} \prod_{b_1:C} (b_0 = b_1) \rightarrow (I \rightarrow C)$$

In Coq, we would write

Definition `rec_I` {C : Type} {b0 b1 : C} (s : (b0 = b1)) : interval → C.

Proof.

```
refine (interval_ind (fun (x : interval) => C) b0 b1 _).
exact ((transport_const seg b1) @ s).
```

Defined.

This means that to define a function from the interval to a type C we only need to provide two endpoints in C and a path between those endpoints. Naturally, we want write computation rules for this recursion principle. These can be obtained from the computation rules of ind_I .

For the point-level rule, we have

$$\text{rec}_I C b_0 b_1 s 0_I \equiv \text{ind}_I C b_0 b_1 ((\text{transport_const } C \text{ seg } b_0) \cdot s) 0_I \equiv b_0$$

. The computation rule for the image of 1_I is totally analogous. These are β -conversion rules just like the ones for any ordinary inductive type, and are purely judgemental.

We would like to have a similar β -conversion rule on the path level, that tells us that $\text{ap}_{\text{rec}_I C b_0 b_1 s} \text{seg}$ is equal to s . We can achieve that only propositionally.

Definition `interval_rec_beta_seg` {C : Type} {b0 b1 : C} (s : b0 = b1) : ap (interval_rec' s)

seg = s.

Proof.

```
pose (p := interval_ind_beta_seg (fun _ => C) b0 b1 (transport_const seg b0 @ s)).
pose (q := apD_const (interval_rec' s) seg).
exact (cancell (transport_const seg b0) (ap (interval_rec' s) seg) s (q @ p)).
```

Defined.

We want to prove $\text{ap}(\text{rec}_I C b_0 b_1 s) \text{seg} = s$. What the induction principle gives us is

$$\text{apd}(\text{rec}_I C b_0 b_1 s) \text{seg} = \text{transport_const } C \text{ seg, } b_0 \cdot s$$

But $\text{apd}(\text{rec}_I C b_0 b_1 s) \text{seg}$ is related to $\text{ap}(\text{rec}_I C b_0 b_1 s) \text{seg}$, through $\text{apd_const}(\text{rec}_I C b_0 b_1 s) \text{seg}$, which has type

$$\text{apd}(\text{rec}_I C b_0 b_1 s) \text{seg} = (\text{transport_const } C \text{ seg } b_0) \cdot (\text{ap}(\text{rec}_I C b_0 b_1 s) \text{seg})$$

Putting these two equalities together and cancelling $(\text{transport_const } C \text{ loop } b)$ on both sides yields the desired element.

Theorem 7.1.2. *The interval is contractible with center 0_I , i.e. $\prod_{x:I} 0 = x$ is inhabited.*

Proof. Definition `interval_is_contractible` : {b : interval & forall (x : interval), b = x}.

Proof.

```
refine (exist _ zero _).
refine (interval_ind (fun x => zero = x) (reflexivity zero) seg _).
refine (transport_paths_r seg (reflexivity zero) @ _).
exact (concat_1p seg)
```

Defined.

We apply interval induction. For the image of 0_I we can provide refl_{0_I} and for the image of 1_I we have seg . The goal then becomes the image of seg , which should be an element of type $\text{refl}_{0_I} \stackrel{\lambda x. 0_I = x}{=} \text{seg}$. This is achievable by instantiating the lemmas `concat_1p` and `transport_paths_r` from previous sections. $\square$

7.2. The Circle.

Definition 7.2.1. We have a higher inductive type, C , with constructors

- $\mathbf{base} : \mathbb{S}^1$
- $\mathbf{loop} : \mathbf{base} = \mathbf{base}$

, and induction principle

$$\mathbf{ind}_{\mathbb{S}^1} : \prod_{P : \mathbb{S}^1 \rightarrow \mathcal{U}} \prod_{b : P \mathbf{base}} ((\mathbf{transport}^P \mathbf{loop} b) = b) \rightarrow \prod_{x : \mathbb{S}^1} P x$$

Additionally, we have the judgemental equality

$$(\mathbf{ind}_{\mathbb{S}^1} P b l) \mathbf{base} \equiv b$$

, as well as the propositional equality

$$_ : \mathbf{apd} (\mathbf{ind}_{\mathbb{S}^1} P b l) \mathbf{loop} = l$$

Just like for the interval, we can derive the circle's recursion principle from its induction principle.

$$\mathbf{rec}_{\mathbb{S}^1} : \prod_{C : \mathcal{U}} \prod_{b : C} (b = b) \rightarrow (\mathbb{S}^1 \rightarrow C)$$

Its computation rules are the following.

$$\mathbf{rec}_{\mathbb{S}^1} C b l \mathbf{base} \equiv b$$

$$_ : \mathbf{ap}_{\mathbf{rec}_{\mathbb{S}^1} C b l} \mathbf{loop} = l$$

The recursion principle can be thought of as the existence part of a *universal property of the circle*. The uniqueness part would correspond to saying that if two functions $f, g : \mathbb{S}^1 \rightarrow A$ and coincide on $\mathbf{base}$ and $\mathbf{loop}$ then they are the same.

Of course, when doing Homotopy Type Theory one has to be careful about what one means by words like “coincide” and “same”. Saying “ f and g coincide on $\mathbf{base}$ ” means that we have $p : f \mathbf{base} = g \mathbf{base}$. But we can't translate their coinciding on $\mathbf{loop}$ as there being a (two-dimensional) path $q : \mathbf{ap}_f \mathbf{loop} = \mathbf{ap}_g \mathbf{loop}$, since $\mathbf{ap}_f \mathbf{loop} : f \mathbf{base} = f \mathbf{base}$ and $\mathbf{ap}_g \mathbf{loop} : g \mathbf{base} = g \mathbf{base}$, which are different types. However, we can express it as there being $q : \mathbf{ap}_f \mathbf{loop} =_{\lambda x. x = x}^p \mathbf{ap}_g \mathbf{loop}$. The statement “ f and g are the same” means there is a homotopy between these two functions.

With that said, we can prove the following uniqueness theorem.

```
Definition uniqueness_circle {A : Type} (f g : Circle -> A) (p : f base = g base)
(q : transport (fun (x : A) => x = x) p (ap f loop) = (ap g loop)) :
forall (x : Circle), f x = g x.
```

Proof.

```
  refine (Circle_ind (fun x => f x = g x) p _).
  rewrite (transport_paths_FlFr loop p).
  rewrite (whiskerL ((ap f loop)^ @ p) (q^ @ (transport_paths_lr p (ap f loop))))).
  rewrite (concat_pp_pp ((ap f loop)^ @ p) (p^ @ ap f loop) p).
  rewrite (concat_pp_p (ap f loop)^ p (p^ @ ap f loop)).
  rewrite (whiskerL (ap f loop)^ (concat_p_Vp p (ap f loop))).
  exact (whiskerR (concat_Vp (ap f loop)) p @ (concat_1p p)).
```

Defined.

We can package the universal property nicely in an equivalence, as follows.

Theorem 7.2.2. *For any $A : \mathcal{U}$, we have an equivalence*

$$(\mathbb{S}^1 \rightarrow A) \simeq \sum_{x \in A} (x = x)$$

Proof. Definition univ_prop_circle ‘{Funext}(A : Type):

(Circle → A) <~> {x : A & x = x}.

Proof.

```
pose (f := fun (h : Circle → A) => exist (fun x => x = x) (h base) (ap h loop)).
pose (g := sig_ind (fun _ => Circle → A) (Circle_rec A)).
refine (Build_Equiv (Circle → A) {x : A & x = x} f (isequiv_adjointify f g _)).
* intro y.
  induction y as (x,p).
  refine (path_sigma(fun x => x = x) _ _ (reflexivity x) _).
  exact (Circle_rec_beta_loop A x p).
* intro h.
  apply path_arrow.
  refine (uniqueness_circle (g (f h)) h (reflexivity (h base)) _).
  exact (Circle_rec_beta_loop A (h base) (ap h loop)).
```

Defined.

□

At first glance, the circle looks like the unit type, only with an explicit loop. However, the circle being contractible would destroy the analogy we are making with topology. Let us then attempt to prove that the circle is contractible, and see what obstacles we find.

We want to define a function of type $\prod_{x:\mathbb{S}^1} \mathbf{base} = x$ (since $\mathbf{base}$ is the only point in the circle we have to work with). For that, we naturally used the induction principle, with $P \equiv \lambda x. \mathbf{base} = x$. For the image of $\mathbf{base}$, we have two options, $\mathbf{refl}_{\mathbf{base}}$ and $\mathbf{loop}$.

If we choose $\mathbf{refl}_{\mathbf{base}}$, then we would have to provide an element of $\mathbf{refl}_{\mathbf{base}} =_{\mathbf{loop}}^P \mathbf{refl}_{\mathbf{base}}$. Unfolding this type with $\mathbf{transport_paths_r}$, this amounts to producing an element of $\mathbf{loop}^{-1} \cdot \mathbf{refl}_{\mathbf{base}} = \mathbf{refl}_{\mathbf{base}}$, which we have no obvious way of doing. If we instead chose $\mathbf{loop}$, then we would need an element of $\mathbf{loop}^{-1} \cdot \mathbf{loop} = \mathbf{loop}$, which we also have no way of doing.

We see that the extra requirement of the induction principle of the circle, relative to the unit type (providing the “image” of $\mathbf{loop}$) prevents us from proving the circle is contractible. In fact, univalence allows us to prove that the circle is not even a set.

Theorem 7.2.3. *If $\mathbf{loop} = \mathbf{refl}_{\mathbf{base}}$, then every type is a set.*

Proof. Definition loop_eq_refl_implies_is_set_Type :

loop = reflexivity base → forall (A : Type), IsTrunc 0 A.

Proof.

```
intros H A.
apply istrunc_S.
intros x y.
apply hprop_allpath.
induction x0.
intro l.
exact((ap (ap (Circle_rec l)) H)~ @ (Circle_rec_beta_loop l)).
```

Defined.

□

This theorem means that if there are types that are not sets, the circle is one of them. But in particular the universe $\mathcal{U}_0$ is not a set, as proved in subsection 6.2.

8. COMPUTING THE FUNDAMENTAL GROUP OF THE CIRCLE

8.1. The Integers. There are many ways to define the integers, $\mathbb{Z}$, from the natural numbers. In [5, Section 6.10], they are defined as a quotient of $\mathbb{N} \times \mathbb{N}$. In the HoTT library, they are defined as an inductive type with constructors $0 : \mathbb{Z}$, $\text{posS} : \mathbb{N} \rightarrow \mathbb{Z}$ and $\text{negS} : \mathbb{N} \rightarrow \mathbb{Z}$. Here, we won't concern ourselves with the particular definition being used (they are all equivalent and produce a suitable notion of integers). It is enough to know that the type $\mathbb{Z}$ has the following features:

- There are functions $\text{succ}_{\mathbb{Z}}, \text{pred}_{\mathbb{Z}}, \text{neg}_{\mathbb{Z}} : \mathbb{Z} \rightarrow \mathbb{Z}$
- $\text{succ}_{\mathbb{Z}}$ and $\text{pred}_{\mathbb{Z}}$ are quasi inverses, and $\text{neg}_{\mathbb{Z}}$ is an autoequivalence
- There is an element

$$\text{ind}_{\mathbb{Z}} : \prod_{P : \mathbb{Z} \rightarrow \mathcal{U}} (P\,0) \rightarrow \left(\prod_{n : \mathbb{N}}, P\,n \rightarrow P(\text{succ}_{\mathbb{Z}}\,n) \right) \rightarrow \left(\prod_{n : \mathbb{N}}, P(\text{neg}_{\mathbb{Z}}\,n) \rightarrow P(\text{pred}_{\mathbb{Z}}(\text{neg}_{\mathbb{Z}}\,n)) \right) \rightarrow \prod_{x : \mathbb{Z}} P\,x$$

[5, Lemma 6.10.12], satisfying the following propositional computation rules⁸:

- $\text{ind}_{\mathbb{Z}}\,P\,d_0\,d_p\,d_n\,0 = d_0$
- $\text{ind}_{\mathbb{Z}}\,P\,d_0\,d_p\,d_n(\text{succ}_{\mathbb{Z}}\,n) = d_p\,n(\text{ind}_{\mathbb{Z}}\,P\,d_0\,d_p\,d_n\,n)$
- $\text{ind}_{\mathbb{Z}}\,P\,d_0\,d_p\,d_n(\text{pred}_{\mathbb{Z}}(\text{neg}_{\mathbb{Z}}\,n)) = d_n\,n(\text{ind}_{\mathbb{Z}}\,P\,d_0\,d_p\,d_n(\text{neg}_{\mathbb{Z}}\,n))$

8.2. The fundamental group. The aim of this section is to prove that $\mathbf{base} = \mathbf{base}$, the loop space of the circle at the base point, is equivalent to $\mathbb{Z}$. Note that $\mathbf{base} = \mathbf{base}$ is a very natural candidate for a “fundamental group”, since equality is defined up to two-dimensional paths. We will use the encode-decode method, just like in subsection 5.3.

To define code , we use circle induction. Its value on $\mathbf{base}$ is what we are aiming to prove the loop space of the circle to be, i.e., the integers. We now need to provide $\text{ap}_{\text{code}}\,\text{loop}$, which should be a path $\text{Int} = \text{Int}$, that expresses the idea that going around the circle once corresponds to adding one in the fundamental group. A good candidate is the path that results from applying univalence to the equivalence succ .

Our goal now will be to construct a family of equivalences $\prod_{x : \mathbb{S}^1} (\mathbf{base} = x) \simeq (\text{code}\,x)$. For encoding, we want to take a path $p : \mathbf{base} = x$ in the circle and return an element of $\text{code}\,x$. Transport along p over code gives us a function $\mathbb{Z} \rightarrow \text{code}\,x$, so we only need to pick an element of $\mathbb{Z}$ to apply it to. If we want $\text{encode}\,\text{loop}$ to be 1, then we need an integer k such that $\text{transport}^{\text{code}}\,\text{loop}\,k = 1$. To decide, let us see what the function $\text{transport}^{\text{code}}\,\text{loop}$ does.

Lemma 8.2.1. *For any integer k , we have*

- (1) $\text{transport}^{\text{code}}\,\text{loop}\,k = \text{succ}\,k$
- (2) $\text{transport}^{\text{code}}\,\text{loop}^{-1}\,k = \text{pred}\,k$

Proof. Item (i) is a matter of applying $\text{transport_postcompose}$ and the propositional computation rules of circle induction and univalence, in the following way.

Definition `trans_code_loop` : `transport code loop == int_succ`.

Proof.

```
intro x.
refine ((transport_postcompose idmap code loop x) @ _).
pose (r := Circle_rec_beta_loop Type Int (path_universe eq_succ)).
refine (((happly (ap (transport idmap) r))) x @ _).
```

⁸these elements are not in the library in this exact form. However, they are defined in the accompanying Coq file.

`exact (happly (transport_idmap_path_universe eq_succ) x).`
 Defined.

Item (ii) follows from (i) if we take the inverses of both sides of the equality, using some path algebra.

Definition `trans_code_loop_inv` : `transport code loop~ == int_pred`.

Proof.

```
intro x.
pose (H0 := (trans_code_loop (transport code loop~ x))~).
pose (H1 := ap int_pred H0).
refine ((int_succ_pred (transport code loop~ x))~ @ H1 @ _).
exact (ap int_pred (transport_pV code loop x)).
```

Defined.

□

Hence, to have `transportcode loop k = 1` we should pick $k = 0$, and now `encode` is defined.

We can define `decode` : $\prod_{x:\mathbb{S}^1} \text{code } x \rightarrow \mathbf{base} = x$ through circle induction. For the image of `base`, we need a function that maps an integer to a path in `base = base`. This in turn can be defined by induction on the integers, in the following way.

Definition `decode_base` := `Int_ind (fun _ => base = base) (reflexivity base) (fun n l => l @ loop) (fun n l => l @ loop~)`.

I.e., `decode_base k` is “`loopk`”. The following lemma looks almost identical to the definition, and its proof is just a matter of using the computational rules for integer induction and switching between the natural numbers and integers.

Lemma 8.2.2. *For any integer k , we have*

- i) `decode_base (succ k) = decode_base k • loop`
- ii) `decode_base (pred k) = decode_base k • loop-1`

Proof. We omit the proof here, but it can be found in the accompanying `.v` file

□

To define `decode` on `loop`, we need a path in `decode_base` $\stackrel{\lambda y. \text{code } y \rightarrow \mathbf{base} = y}{=} \text{decode_base}$. We can use `transport_arrow` to unfold the left side of this equality into $\lambda x. \text{transport}^{\lambda y. \mathbf{base} = y} \text{loop} (\text{decode_base}(\text{transport}^{\text{code}} \text{loop}^{-1} x))$. Then, we can use function extensionality to transform the goal from an equality of functions to a homotopy. If we introduce $k : \mathbb{Z}$, the goal then becomes:

$$\text{transport}^{\lambda y. \mathbf{base} = y} \text{loop} (\text{decode_base}(\text{transport}^{\text{code}} \text{loop}^{-1} k)) = \text{decode_base } k$$

Which by `transport_paths_r` is

$$\text{decode_base} (\text{transport}^{\text{code}} \text{loop}^{-1} k) \bullet \text{loop} = \text{decode_base } k$$

But now we have by lemma 8.2.1 that `transportcode loop-1 k = pred k`, so the goal becomes simply

$$\text{decode_base} (\text{pred } k) \bullet \text{loop} = \text{decode_base } k$$

Which after a simple manipulation can be turned into precisely the statement of lemma 8.2.2. Thus, `decode` is defined. Formally:

Definition decode : forall {x : Circle}, code x -> base = x.

Proof.

```

  refine (Circle_ind (fun x => code x -> base = x) decode_base _).
  refine (transport_arrow code (fun x => base = x) loop(decode_base) @ _).
  apply path_arrow.
  intro k.
  refine (transport_paths_r loop (decode_base (transport code loop^ k)) @ _).
  rewrite (trans_code_loop_inv k).
  refine (cancelR ((decode_base) k.-1%int @ loop) ((decode_base) k) loop^ _).
  refine (concat_pp_V ((decode_base) k.-1%int) loop @ _).
  exact (decode_base_pred k).

```

Defined.

Now all we need to do is to prove both compositions are homotopic to the respective identities. Just like in the case of co-products, proving that, for $x \in \mathbb{S}^1$, $(\text{decode } x) \circ (\text{encode } x) \sim \text{id}_{\text{base}=x}$ is trivial after path induction.

The other homotopy requires a little bit more work. We will define $\text{encode_decode} : \prod_{x:\mathbb{S}^1} (\text{encode } x) \circ (\text{decode } x) \sim \text{id}_{\text{code } x}$ by circle induction. For that we need a homotopy $\text{encode_decode_base}(\text{encode base}) \circ (\text{decode base}) \sim \text{id}_{\text{code base}}$. Thankfully, since $\mathbb{Z}$ is a set, we won't need to supply any higher paths.

Let us then focus on $\text{encode_decode_base}$. We can prove it by integer induction, as follows.

Definition encode_decode_base : forall k : code base, encode (decode k) = k.

Proof.

```

  intro k.
  unfold encode.
  induction k.
  * reflexivity.
  * refine (ap encode (decode_base_succ n) @ _).
    refine (transport_pp code (decode_base n) loop zero @ _).
    refine (trans_code_loop (transport code (decode_base n) zero) @ _).
    exact (ap int_succ IHk).
  * refine (ap encode (decode_base_pred (int_neg n)) @ _).
    refine (transport_pp code (decode_base (int_neg n)) loop^ zero @ _).
    refine (trans_code_loop_inv (transport code (decode_base (int_neg n)) zero) @ _).
    exact (ap int_pred IHk).

```

Defined.

Putting all these ingredients together, we arrive at the family of equivalences:

$$\text{circle_paths} : \prod_{x:\text{Circle}} (\text{base} = x) \simeq (\text{code } x)$$

And applying circle_paths to base . we get an element of $(\text{base} = \text{base}) \simeq \mathbb{Z}$, which is what we wanted.

9. CONCLUSION

We have developed the basic constructions of Homotopy Type Theory and their formalization in Coq. In particular, we've been able to represent interesting topological spaces such as the circle in a synthetic, type-theoretic fashion and computed its fundamental group. Throughout this process, we have formalized proofs that are not yet in the library.

REFERENCES

- [1] Andrej Bauer et al. *The HoTT Library: A formalization of homotopy type theory in Coq*. <https://github.com/HoTT/Coq-HoTT>. Accessed: 2026-04-28. 2016.
- [2] Herbert B. Enderton. *Elements of Set Theory*. Academic Press, 1977.
- [3] Per Martin-Löf. *Intuitionistic Type Theory*. Bibliopolis, 1984.
- [4] The Coq Development Team. *The Coq Proof Assistant Reference Manual, Version 8.19.0*. Accessed: 2026-04-28. Inria. 2024. URL: <https://rocq-prover.org/doc/V8.19.0/refman/index.html>.
- [5] The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study: <https://homotopytypetheory.org/book>, 2013.